

CultiKure: An AI-Powered Web Application for Plant Disease Detection and Cure Recommendation

Final Year Project



Supervisor: DR. Syed Ali Nawaz Shah

Presented by:

Student Name
Student Reg#

Abdullah Javeed
F22BINFT1M01076



Department of Information Technology (DIT)

The Islamia University of Bahawalpur

CultiKure: An AI-Powered Web Application for Plant Disease Detection and Cure Recommendation

By

Abdullah Javeed (F22BINFT1M01076)

Project submitted to

Department of Information Technology,

The Islamia University of Bahawalpur.

in partial fulfillment of the requirements for the degree of

**BACHELOR OF SCIENCE
IN
INFORMATION TECHNOLOGY (BSIT)**

Supervisor Signature	In- Charge Examination Signature
Chairman, Department of Information Technology	

Abstract

Agriculture forms the absolute backbone of Pakistan's economy, yet the sector consistently suffers massive financial losses annually due to the delayed, subjective, and often inaccurate identification of crop diseases. This critical issue is significantly worsened by a severe shortage of accessible agronomic experts, limited laboratory testing facilities in rural areas, and persistent language barriers that prevent small-scale farmers from obtaining timely diagnostic support. The core problem this project aims to solve is to eliminate the reliance on slow manual crop inspections by providing an automated, highly accurate, and centralized diagnostic platform. Specifically, our work focuses on developing a solution that is strictly tailored for end-users with limited technical literacy, completely removing the need for high-end hardware or native mobile application installations.

To resolve this challenge, we engineered CultiKure, a fully responsive, API-first web application that functions as a real-time digital agronomist. Our technical approach leverages a highly efficient dual-API integration strategy: we utilized the **Plant.id** API powered by advanced Convolutional Neural Networks (CNNs)—to perform precise, computer-vision-based image analysis for instant disease detection. Following the visual diagnosis, the system orchestrates with the **Google Gemini** API to generate comprehensive, contextually relevant treatment plans comprising both organic and chemical remedies. To extend the extent of our work beyond basic diagnostics, we integrated keyless auto-location services for real-time environmental context tracking, built an interactive AI chat interface, and implemented automated **PDFMake** lab report generation within a secure, Flask-driven backend architecture.

The results of this implementation demonstrate that CultiKure successfully reduces the diagnostic turnaround time from several days to under ten seconds. It delivers highly accurate pathology identification and actionable prescriptions directly to any internet-enabled device, vastly outperforming traditional methods in both speed and accessibility. In conclusion, this project provides a highly scalable and significant technological intervention that democratizes access to expert agricultural knowledge. By empowering rural farming communities with an instant, robust, and free diagnostic tool, CultiKure facilitates precise agricultural decision-making and serves as a vital step toward securing and maximizing crop yields nationwide.

Acknowledgement

First and foremost, all praises and thanks are to Almighty Allah, the Most Merciful and the Most Beneficent, for granting me the strength, knowledge, and perseverance required to successfully complete this Final Year Project. All respect and blessings be upon the Holy Prophet Muhammad (Peace Be Upon Him), who is the greatest source of knowledge and guidance for humanity.

I would like to express my deepest and most sincere gratitude to my respected supervisor, **DR. Syed Ali Nawaz Shah**, for his unwavering support, continuous encouragement, and invaluable technical guidance throughout the development of CultiKure. His profound insights and constructive feedback were instrumental in shaping this project from a mere concept into a fully functional, AI-driven application. His patience and mentorship have been a constant source of inspiration during this entire journey. I am also highly thankful to the esteemed faculty members and the administration of the **Department of Information Technology at The Islamia University of Bahawalpur**, for providing a highly conducive academic environment, state-of-the-art resources, and a strong foundational knowledge base that made this technological endeavor possible.

Last but certainly not least, I owe a profound debt of gratitude to my beloved parents and my dearest friends. Their endless prayers, moral support, and unconditional love kept me motivated through all the challenging phases and sleepless nights of this development cycle. This achievement would not have been possible without their continuous encouragement.

List of Figures

<u>Figure #</u>	<u>Figure Tittle</u>	<u>Page #</u>
Figure 1	Figure 1 - Use Case Diagram	7
Figure 2	Figure 2 - ER Diagram	9
Figure 3	Figure 3 - System Sequence Diagram	10
Figure 4	Figure 4 - High-Level System Architecture Diagram	11
Figure 5	Figure 5 - System Workflow Activity Diagram	12

List of Tables

<u>Table #</u>	<u>Table Title</u>	<u>Page #</u>
Table 1	Table 1 - UC-1	3
Table 2	Table 2 - UC-1	4
Table 3	Table 3 - UC-1	5
Table 4	Table 4 - UC-1	5
Table 5	Table 5 - UC-1	6
Table 6	Table 6 - UC-1	7
Table 7	Table 7: TC-1	18
Table 8	Table 8: TC-2	18
Table 9	Table 9: TC-3	19
Table 10	Table 10: TC-4	19
Table 11	Table 11: TC-5	19
Table 12	Table 12: TC-6	20
Table 13	Table 13: Summary of All Test Results	20

TABLE OF CONTENTS

ABSTRACT.....	I
ACKNOWLEDGEMENT.....	I
LIST OF FIGURES.....	III
LIST OF TABLES.....	IV
CHAPTER 1. INTRODUCTION.....	1
1.1 INTRODUCTION	1
1.2 OBJECTIVES	1
1.3 PROBLEM STATEMENT.....	1
1.4 SCOPE	2
CHAPTER 2. REQUIREMENTS ANALYSIS	2
2.1 LITERATURE REVIEW	2
2.2 USER CLASSES AND CHARACTERISTICS	2
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS	2
2.4 ASSUMPTIONS AND DEPENDENCIES.....	3
2.5 FUNCTIONAL REQUIREMENTS	3
2.6 USE CASE DIAGRAM	7
2.7 NONFUNCTIONAL REQUIREMENTS	8
2.8 OTHER REQUIREMENTS	8
CHAPTER 3. SYSTEM DESIGN	9
3.1 APPLICATION AND DATA ARCHITECTURE	9
3.2 COMPONENT INTERACTIONS AND COLLABORATIONS	10
3.3 SYSTEM ARCHITECTURE	10
3.4 ARCHITECTURE EVALUATION.....	11
3.5 COMPONENT-EXTERNAL ENTITIES INTERFACE	12
3.6 SCREENSHOTS/PROTOTYPE	12
3.7 OTHER DESIGN DETAILS.....	17
CHAPTER 4. TEST SPECIFICATION AND RESULTS	17
4.1 TEST CASE SPECIFICATION	17
4.2 SUMMARY OF TEST RESULTS.....	20
CHAPTER 5. CONCLUSION AND FUTURE WORK	21
5.1 PROJECT SUMMARY	21
5.2 PROBLEMS FACED AND LESSONS LEARNED	21
5.3 FUTURE WORK	21
REFERENCES.....	22
APPENDIX A GLOSSARY	23
APPENDIX B DEPLOYMENT/INSTALLATION GUIDE.....	24

Revision History

Name	Date	Reason For Changes	Version
Abdullah Javeed	25-05-2026	Initial Draft including Chapters 1 to 3 and system architectural diagrams.	1.0
Abdullah Javeed	04-06-2026	Finalized Testing (Chapter 4), Conclusion (Chapter 5), References, and Appendices.	2.0

Chapter 1. Introduction

1.1 Introduction

The integration of Artificial Intelligence (AI) in agriculture has significantly transformed traditional farming practices. Historically, plant disease detection relied on visual inspection by experts, which is slow, subjective, and prone to human error. In Pakistan, where the economy heavily relies on the agricultural sector, the industry is continuously hampered by the slow and often inaccurate identification of plant illnesses. This critical issue results in severe financial losses due to underperforming crop yields. Slow manual crop inspections by agricultural specialists, who are generally in short supply, create dangerous delays and allow farmers to lose control of plant health. To fill this gap, this project successfully developed "CultiKure", a highly responsive, centralized web application. CultiKure utilizes an API-first architecture, smoothly integrating the Plant.id API for precise image-based disease detection and the Google Gemini API for contextually relevant prescription generation. Beyond its initial scope, the finalized system has been enhanced with live GPS-based weather integration, dynamic PDF lab report generation, and structured social sharing to serve as a complete, fully automated expert system for rural farming communities.

1.2 Objectives

The final objectives achieved after the successful completion of the CultiKure project are:

- To develop a responsive, centralized web application that is easily accessible on any device without the need for native installation.
- To integrate the Plant.id API for the accurate, real-time automated detection of plant diseases from user-uploaded leaf images.
- To utilize the Google Gemini API to act as an AI agronomist, generating detailed, human-readable information regarding symptoms, causes, and comprehensive cure recommendations (both organic and chemical).
- To implement a secure User Management and History module that persistently stores all diagnostic results in a database for ongoing crop monitoring.
- To provide an intuitive, Glassmorphism-based UI featuring live weather context, voice-to-text input, and one-click PDF lab report generation to facilitate users with limited technical literacy.

1.3 Problem Statement

This project solved the critical problem of massive financial losses in the agricultural sector caused by the delayed and inaccurate identification of crop diseases. Existing mobile solutions presented high-cost barriers, language barriers, and platform restrictions (mobile-only), leaving small-scale farmers without immediate access to agronomic experts. CultiKure is a comprehensive web-based software package that functions as an instant digital agronomist. It applies advanced computer vision and generative AI algorithms to process plant images and output structured treatments. By doing so, it achieves vastly better accessibility and efficiency than its competitors, completely eliminating the need for complex installations or premium subscriptions.

1.4 Scope

The scope of the CultiKure product covers the deployment of a cloud-based web application that acts as a central integration hub for farmers and gardeners. The covered scope includes a responsive frontend interface, a Python/Flask backend system, secure user authentication, a persistent scan history dashboard, and the dual-API integration of Plant.id and Google Gemini for real-time data processing. The system operates entirely online, requiring active internet connectivity to perform diagnostics. Items strictly outside the scope include the development of native mobile applications (Android/iOS), offline diagnostics, custom AI model training from scratch, and the integration of hardware components like IoT sensors or agricultural drones.

Chapter 2. Requirements Analysis

2.1 Literature Review

The integration of Artificial Intelligence in agriculture has gradually shifted plant disease detection from manual, visual inspections to automated Deep Learning techniques. Existing systems in the market, such as Plantix and PictureThis, utilize image recognition to detect pests and deficiencies. However, an analysis of these systems reveals significant limitations: they are primarily native mobile applications (requiring installation and device storage), they often suffer from language barriers, or they operate on "freemium" models that create cost barriers for small-scale farmers. Furthermore, systems like Agrio are overly complex, targeting enterprise farming rather than local, rural agriculture. CultiKure was developed in light of this literature review to bridge these exact gaps. By adopting an API-first, web-based approach, CultiKure eliminates the need for installation. Furthermore, by integrating Google Gemini API, it moves beyond static databases to provide context-aware, generative AI explanations and treatment plans (organic and chemical), completely democratizing access to expert knowledge without financial or hardware constraints.

2.2 User Classes and Characteristics

We have identified two primary user classes for CultiKure:

- **Farmers and Home Gardeners (End Users):** This favored user class ranges from professional agriculturalists to hobbyist gardeners. Their technical literacy varies from low to moderate. Consequently, they require a highly intuitive, simple, and responsive Glassmorphism-based User Interface (UI) that demands minimal training. Their main interactions involve registering, capturing/uploading leaf images, and reading generated AI diagnostics.
- **System Administrator:** The super-user responsible for the overall maintenance and operational integrity of the web application. This user possesses high technical proficiency, interacts with the Python (Flask) backend, monitors third-party API usage (Plant.id, Gemini), and manages database security.

2.3 Design and Implementation Constraints

The development of CultiKure was guided by several constraints:

- **Platform Limitation:** The system is strictly a web application (HTML/CSS/JS frontend with a Flask backend) and does not have standalone native mobile versions (.apk or .ipa).
- **Memory/Hardware Constraints:** To ensure efficient API processing and reduce server load, user-uploaded images are restricted to a maximum size of 5MB.
- **External API Dependency:** The system's core diagnostic logic is constrained by the performance and response formats of third-party interfaces, specifically the Plant.id Health Assessment API and Google Gemini API.

2.4 Assumptions and Dependencies

- **Connectivity Assumption:** It is assumed that the end-user has an active and stable internet connection, as the system requires real-time cloud processing to bridge user requests with external AI engines.
- **Third-Party Dependencies:** CultiKure is heavily dependent on external services, including the Plant.id API for visual processing, the Google Gemini API for knowledge generation, and Open-Meteo/IPAPI for real-time weather tracking without API keys. If these external APIs change their endpoint structures or experience downtime, the CultiKure backend must handle the errors gracefully.
- **Database Dependency:** The system depends on a lightweight database (SQLite during development) for storing user profiles and persistent scan histories.

2.5 Functional Requirements

2.5.1 Name of Use-Case 1

Identifier		UC-1
Purpose		To allow new users (Farmers, Gardeners) to create an account and verify their identity to access the system.
Priority		High
Pre-conditions		The user must have a stable internet connection and a valid email address.
Post-conditions		A new user profile is created, the email is verified via OTP, and the user is redirected to the Login page.
Typical Course of Action		
S#	Actor Action	System Response
1	User navigates to the registration page.	System displays the registration form.
2	User enters Full Name, Email Address, Password, and Confirm Password.	System validates the inputs (email format, password length, and match).
3	After Validation	System generates and sends an OTP to the user's provided email address for verification.
4	User enters the received OTP.	System verifies the OTP, creates a new entry in the user database, and redirects to the login interface.
Alternate Course of Action		
S#	Actor Action	System Response
1	User clicks the "Login" link (if already registered).	System redirects the user to the authentication page.
2	User provides incorrect email format or passwords do not match.	System displays an error message
3	User enters an incorrect OTP.	System displays an "Invalid OTP" error and prompts the user to try again.

Table 1: UC-1

2.5.2 Name of Use-Case 2

Identifier		UC-2
Purpose		To authenticate registered users and grant them access to the system's core features.
Priority		High
Pre-conditions		The user must have a previously registered and verified account in the system database.
Post-conditions		The user is granted access to their private dashboard and scan history.
Typical Course of Action		
S#	Actor Action	System Response
1	User opens the login page and enters their registered Email Address and Password.	System compares the entered credentials with the records in the database.
2	After successful evaluation of credentials	On success, the system creates a secure session and redirects the user to the main Dashboard.
Alternate Course of Action		
S#	Actor Action	System Response
1	User enters an incorrect email or password.	System displays an "Invalid Credentials" error message.

Table 2: UC-2

2.5.3 Name of Use-Case 3: Upload Plant Image for Diagnosis

Identifier		UC-3
Purpose		To enable users to submit a digital image of a diseased plant leaf for real-time AI analysis.
Priority		High
Pre-conditions		The user must be logged into the system and have a clear image of the infected plant leaf.
Post-conditions		The image is successfully processed by the Plant.id API, and the system prepares for the diagnosis display.
Typical Course of Action		
S#	Actor Action	System Response
1	User clicks the "Upload" button and chooses to capture a photo or select a file.	System checks if the file is a supported format (JPG/PNG) and within the 5MB size limit.
2	After successful evaluation of credentials	System sends the image data securely to the Plant.id API and displays an "Analyzing..." progress indicator.
3	After showing result	System receives the disease identification result and confidence score.

Alternate Course of Action		
S#	Actor Action	System Response
1	User uploads a file exceeding 5MB or an unsupported format.	System displays an error and prompts the user to take a clearer photo or upload a valid file.
2	After it ...	External API (Plant.id) times out due to network issues; system displays a retry prompt.

Table 3: UC-3

2.5.4 Name of Use-Case 4: View Diagnosis and Cure Plan (Updated Features)

Identifier		UC-4
Purpose		To display the disease identification, AI-generated treatment plans, and real-time weather context to the user.
Priority		High
Pre-conditions		The system must have successfully received a response from the Plant.id API.
Post-conditions		The diagnostic results are displayed to the user and automatically saved to their scan history.
Typical Course of Action		
S#	Actor Action	System Response
1	User awaits the final diagnosis after the image analysis is completed.	System passes the detected disease name to the Google Gemini API to fetch contextual, generative cure advice.
2	User grants location permission (if requested by the browser) or relies on IP detection.	System fetches live local weather data using keyless IPAPI & Open-Meteo integration.
3	User views the complete diagnosis and treatment plan.	System displays a comprehensive Glassmorphism result card containing the disease name, Gemini AI symptoms, cure plan, and current weather.
4	After it...	System automatically saves the scan result, date, and image to the database.
Alternate Course of Action		
S#	Actor Action	System Response
1	User clicks the "Generate PDF" button.	System utilizes the pdfmake library to instantly generate and download a HD Lab Report.
2	User clicks the "Share" button.	System opens structured social sharing popups for WhatsApp, Facebook, or Gmail.

Table 4: UC-4

2.5.5 Name of Use-Case 5: View Scan History

Identifier	UC-5	
Purpose	To allow the user to access a chronological log of all their previous plant disease scans and downloaded reports.	
Priority	Medium	
Pre-conditions	The user must have a history of at least one previous scan saved in the database.	
Post-conditions	User can review past reports without re-uploading images.	
Typical Course of Action		
S#	Actor Action	System Response
1	User clicks on the "History" tab on the main navigation dashboard.	System retrieves all scan records associated with the user's ID from the database.
2	Now user can see history records.	System displays a chronological list showing thumbnails of plant images, detected disease names, and scan dates.
3	User clicks a specific entry.	System displays the full diagnosis and Gemini AI cure details exactly as they appeared originally.
Alternate Course of Action		
S#	Actor Action	System Response
1	User attempts to view history but has never performed a scan.	System displays a "No records found" message.

Table 5: UC-5

2.5.6 Name of Use-Case 6: User Logout

Identifier		UC-6	
Purpose		To securely terminate the user's active session and protect account privacy.	
Priority		Medium	
Pre-conditions		The user must be actively logged into the CultiKure system.	
Post-conditions		The session is destroyed, and the user is redirected to the Login page.	
Typical Course of Action			
S#	Actor Action		System Response
1	User clicks the "Logout" button on the dashboard navigation menu.		System safely terminates the active session token and clears temporary cookies.
2	After it...		System redirects the user back to the public Login screen.
Alternate Course of Action			

S#	Actor Action	System Response
1	User simply closes the browser tab.	System automatically expires the session after a set period of inactivity.

Table 6: UC-6

2.6 Use Case Diagram

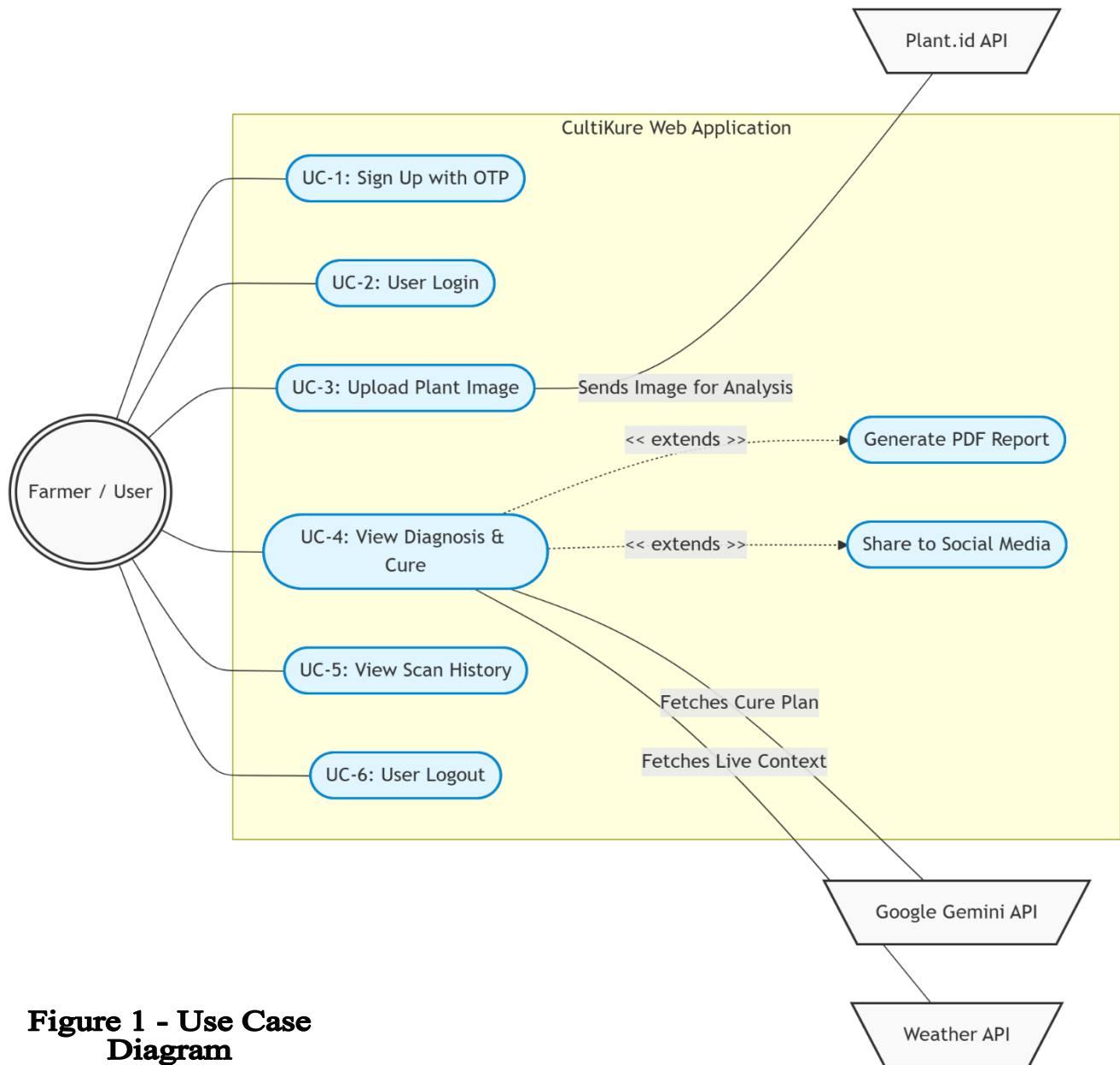


Figure 1 - Use Case Diagram

2.7 Nonfunctional Requirements

2.7.1 Performance Requirements

The system is designed to provide complete diagnosis and cure recommendations within **5 to 10** seconds of image submission, contingent upon the latency of the external APIs and internet speed. The **Flask** backend must handle concurrent requests efficiently, and the system optimizes user-uploaded images (maximum **5MB**) to reduce bandwidth and improve processing speed.

2.7.2 Safety Requirements

The system must ensure that no user data or scan history is corrupted in the event of an unexpected network failure or API timeout. CultiKure is designed to handle external service disruptions gracefully; if the Plant.id or Gemini API fails to respond, the system safely aborts the process and displays a user-friendly error message rather than crashing or exposing sensitive backend stack traces.

2.7.3 Security Requirements

To protect user privacy and prevent unauthorized access, user passwords are not stored in plain text; they are hashed using secure cryptographic algorithms before database storage. All data transmissions between the user's browser, the Flask server, and the external AI services are encrypted using HTTPS/TLS protocols. Furthermore, highly sensitive API keys (Plant.id and Google Gemini) are strictly stored in hidden server environment variables and are never exposed in the client-side source code.

2.7.4 Additional Software Quality Attributes

- **Usability:** The interface is designed utilizing a minimalist Glassmorphism theme specifically crafted for users with limited technical literacy. Critical tasks, such as uploading a plant image and viewing a diagnosis, are streamlined to require minimal clicks, supported by voice-to-text functionalities to assist rural farmers.
- **Portability & Compatibility:** As an API-first web application, CultiKure strictly follows Responsive Web Design principles. It automatically adapts to various screen sizes across smartphones, tablets, and desktop displays, completely bypassing the need for heavy native Android or iOS installations.
- **Maintainability:** The system is built upon a highly modular Python (Flask) backend architecture. This decoupling of the frontend interface from the backend processing allows for future API migrations (e.g., upgrading to newer Gemini models) without disrupting the core system functionality.

2.8 Other Requirements

- **Database Requirements:** The system utilizes a lightweight relational database (SQLite) for the persistent storage of user profiles, authentication credentials, and historical scan records during the current deployment phase.
- **External Interface Requirements:** CultiKure strictly depends on secure HTTPS/REST communications with external cloud-based intelligence engines. This includes the Plant.id API for computer-vision-based leaf assessment, the Google Gemini API for generating detailed organic and chemical treatment plans, and Open-Meteo/IPAPI for fetching real-time environmental data without requiring user API keys.

Chapter 3. System Design

3.1 Application and Data Architecture

This section presents the logical data model of the CultiKure system. The application utilizes a relational database structure to securely manage user profiles and persist their diagnostic scan histories. Below is the Entity-Relationship (ER) Diagram that defines the primary entities (User and ScanHistory) and their cardinality. A single user can have multiple scan records associated with their account.

ER Diagram:

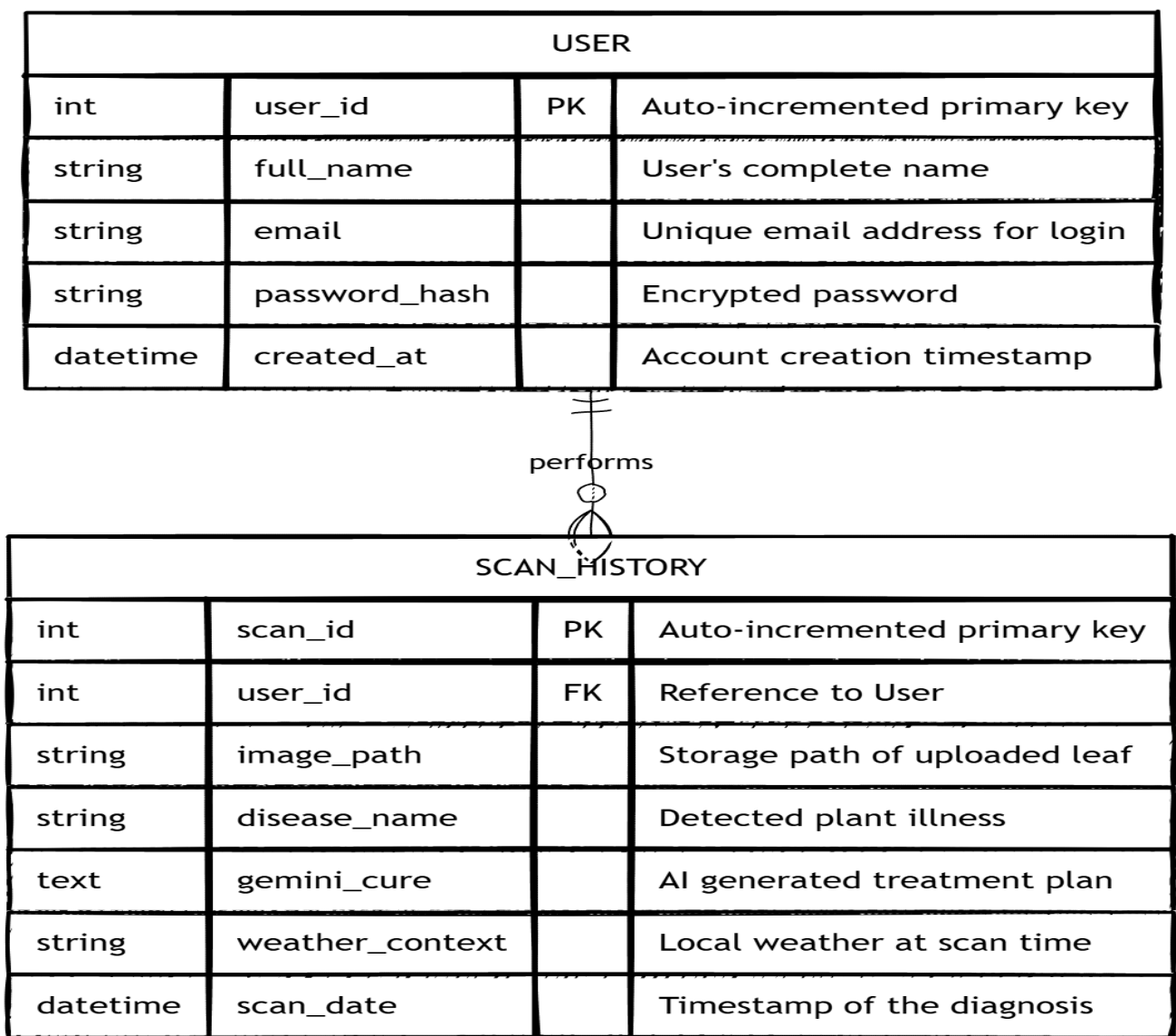


Figure 2 - ER Diagram

3.2 Component Interactions and Collaborations

The component interaction within CultiKure is highly dynamic, relying on asynchronous communication between the client frontend, the Flask backend server, and multiple external APIs. The Sequence Diagram below illustrates the chronological flow of messages and data when a user uploads a leaf image for diagnosis. It highlights the orchestration of the Plant.id API for computer vision, the Google Gemini API for generative prescriptions, and the Weather API for environmental context before delivering the final result to the user.

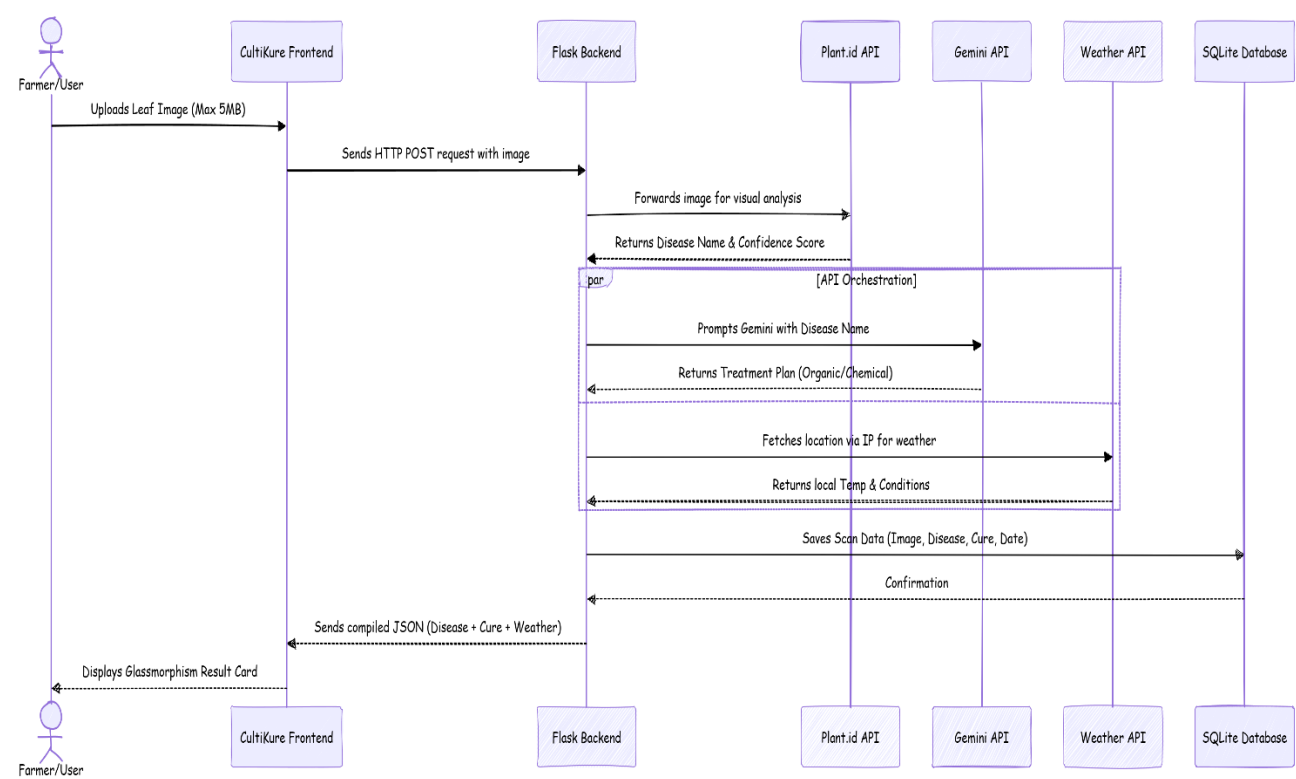


Figure 3 - System Sequence Diagram

3.3 System Architecture

The technical architecture of CultiKure is based on a modern Client-Server API-First model. The system is decoupled into a lightweight frontend interface and a robust Python-based backend that orchestrates external intelligence. The user interacts with the web interface, which sends HTTP/REST requests to the Flask Server. The server handles authentication via SQLite and communicates asynchronously with the Plant.id API, Google Gemini API, and Weather APIs before rendering the compiled diagnostic result back to the client.

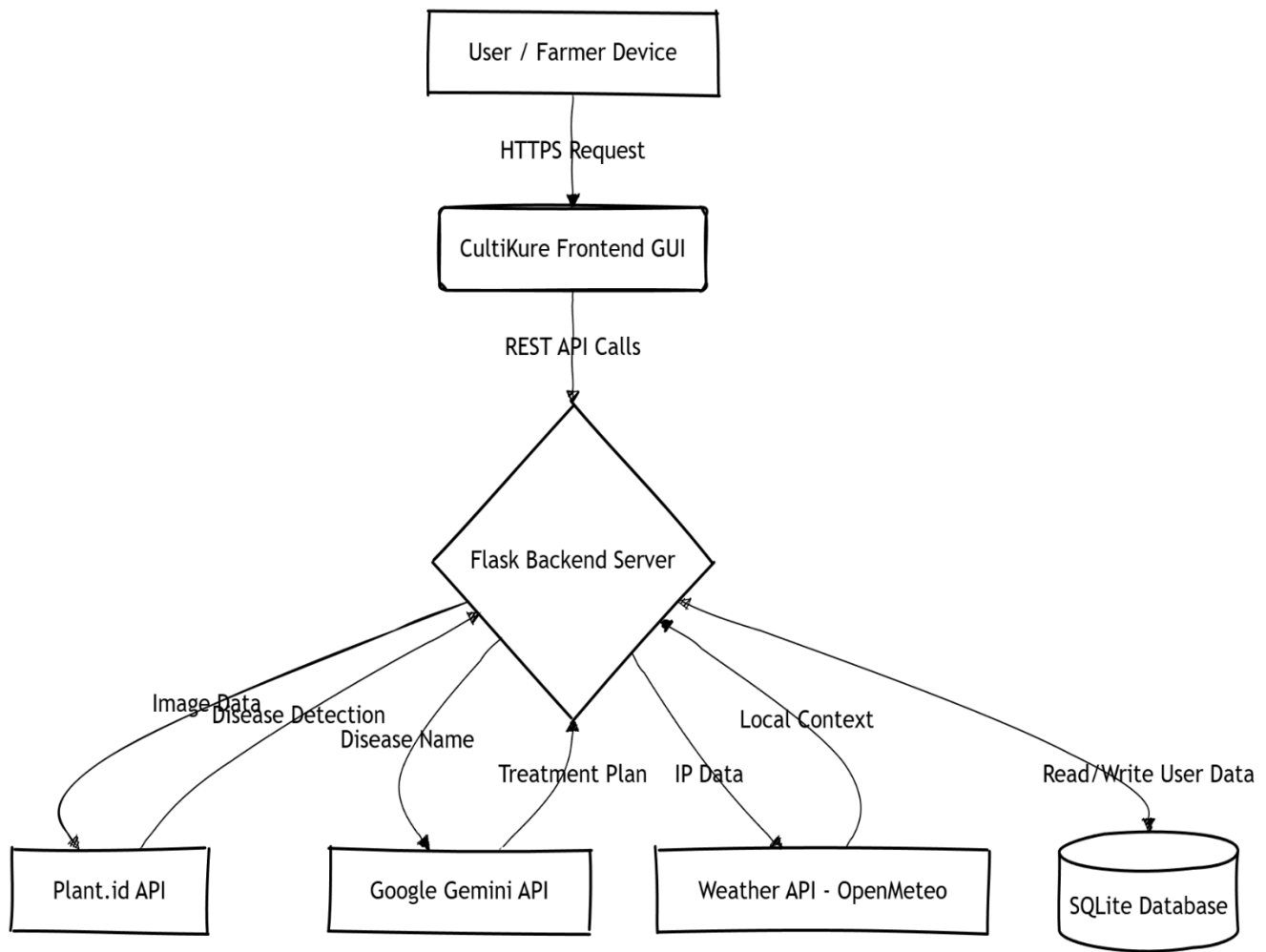


Figure 4 - High-Level System Architecture Diagram

3.4 Architecture Evaluation

The selection of the underlying technology stack was driven by the project's core objectives of accessibility, speed, and maintainability:

- **Web Application vs. Native Mobile App:** A responsive web application was selected over a native Android/iOS application to eliminate hardware, memory, and installation barriers for rural farmers using low-end devices.
- **Python (Flask) Framework:** Flask was preferred over bulkier frameworks (like Django) or PHP because it is highly modular, lightweight, and natively supports the best ecosystem for AI and API integrations.
- **API-First Approach:** Instead of training a local AI model from scratch (which would require immense computational resources and constant updates), the system leverages enterprise-grade models (Plant.id and Gemini). This ensures a significantly higher accuracy rate and provides dynamic, generative advice rather than static database responses.

3.5 Component-External Entities Interface

The communication between CultiKure components and external entities is strictly managed via secure RESTful HTTPS requests.

- **Plant.id API Interface:** The Flask backend acts as a proxy, sending a POST request containing the base64-encoded leaf image to the Plant.id endpoints. The API returns a JSON object containing the name of the disease and a confidence probability score.
- **Google Gemini API Interface:** The backend sends a structured text prompt via a POST request to the Gemini engine, requesting specific chemical and organic cures based on the Plant.id result.
- **Weather API Interface:** The backend retrieves the user's approximate location using a keyless IP API and sends a GET request to Open-Meteo to fetch real-time local temperature and conditions.

3.6 Screenshots/Prototype

3.6.1 Workflow

The complete workflow of the CultiKure system defines the sequential activities performed by the user and the system during a diagnostic cycle.

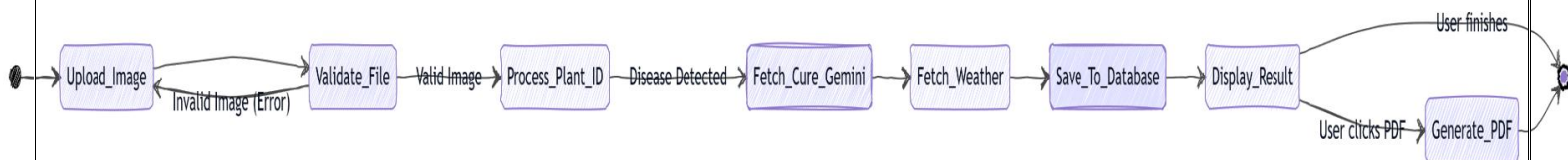
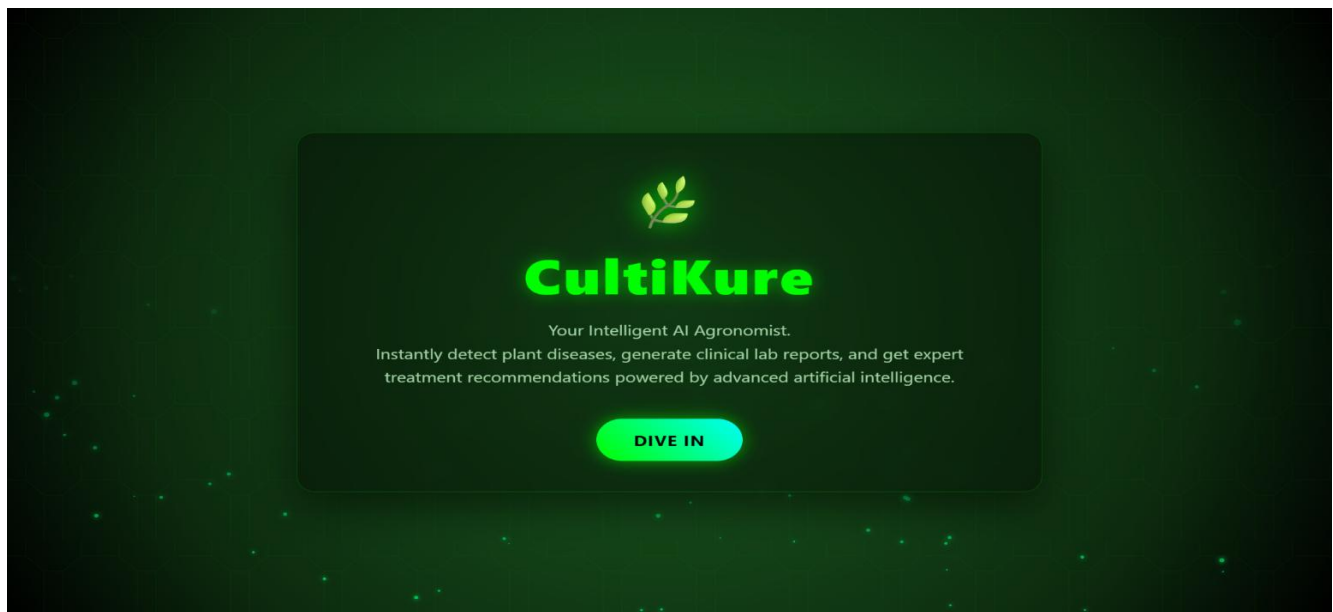
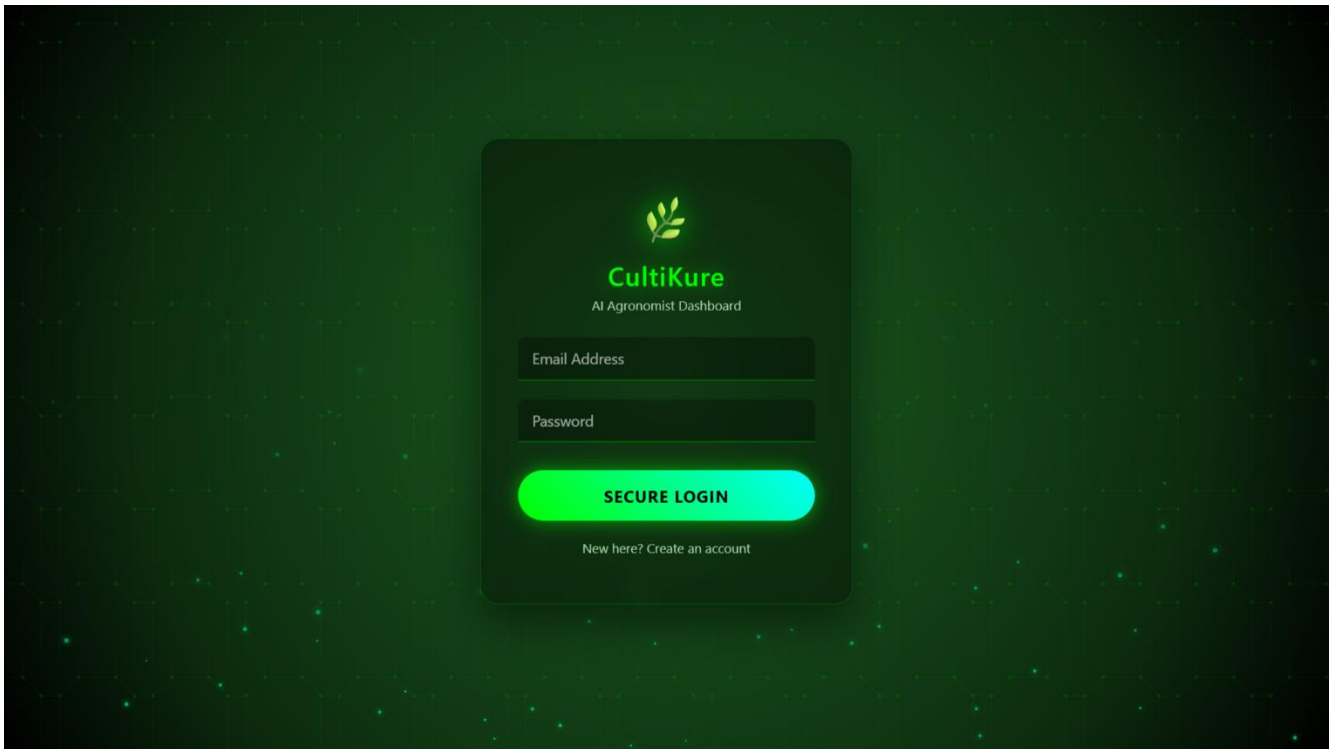


Figure 5 - System Workflow Activity Diagram

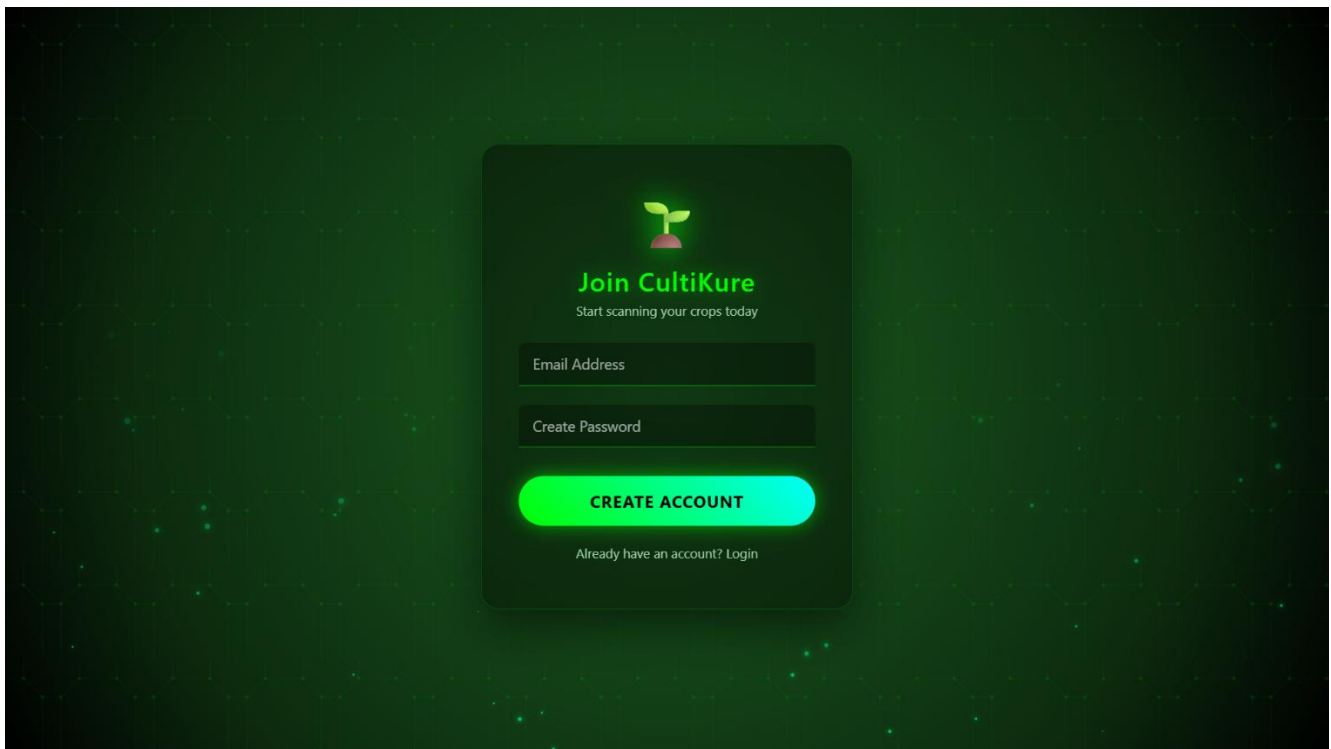
3.6.2 Screens



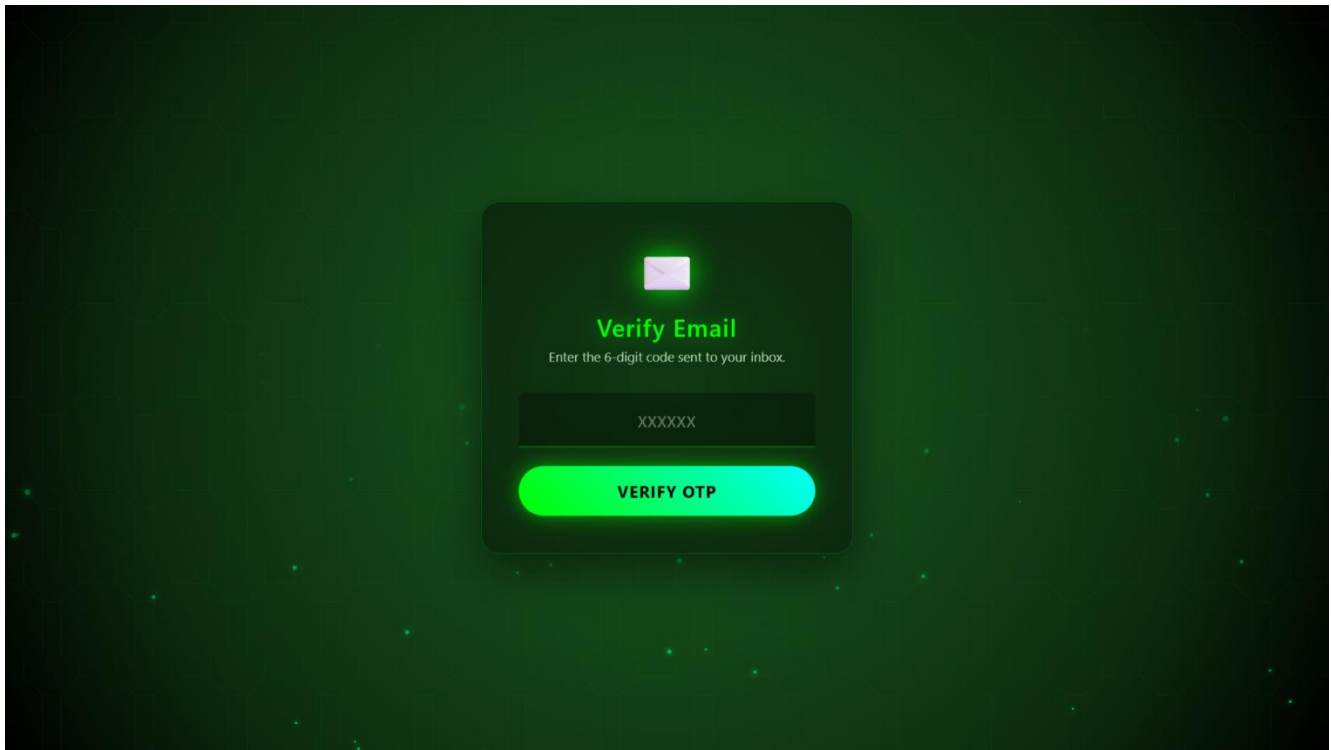
Screen 1 – Landing Page



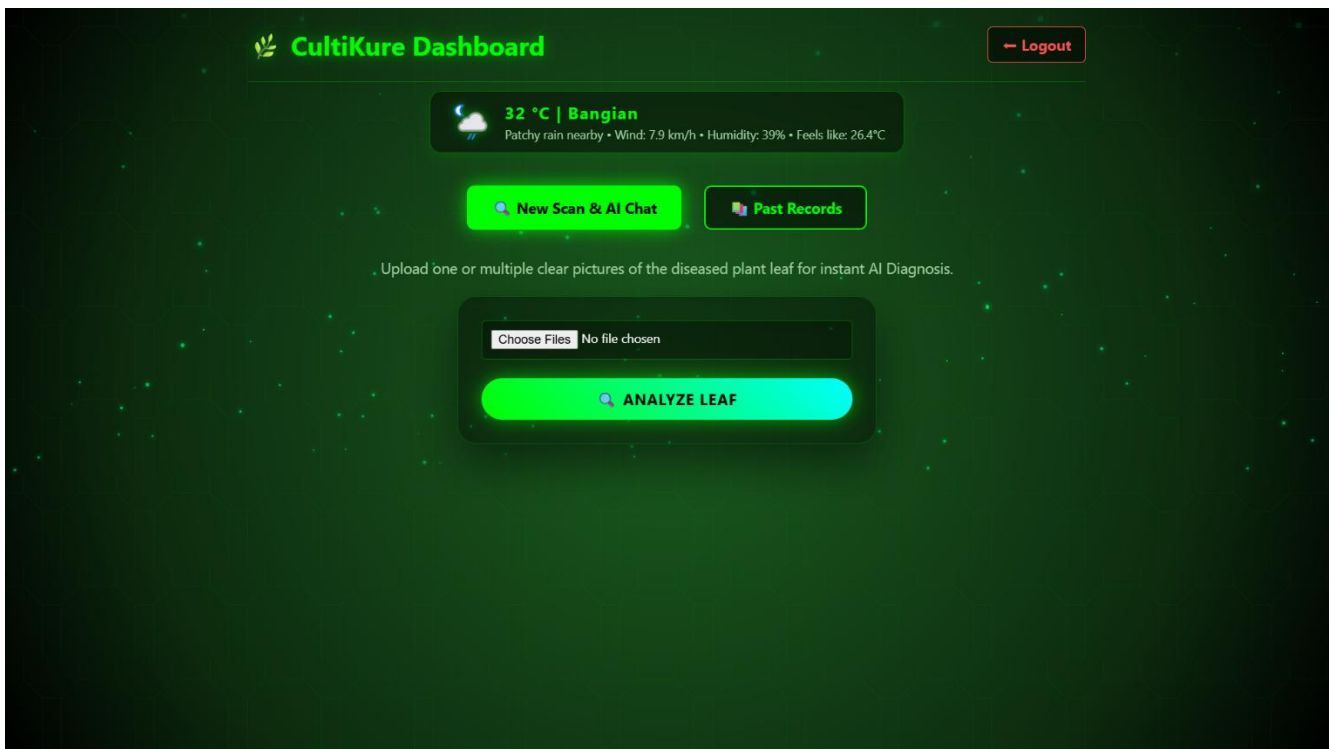
Screen 2 – Login Page



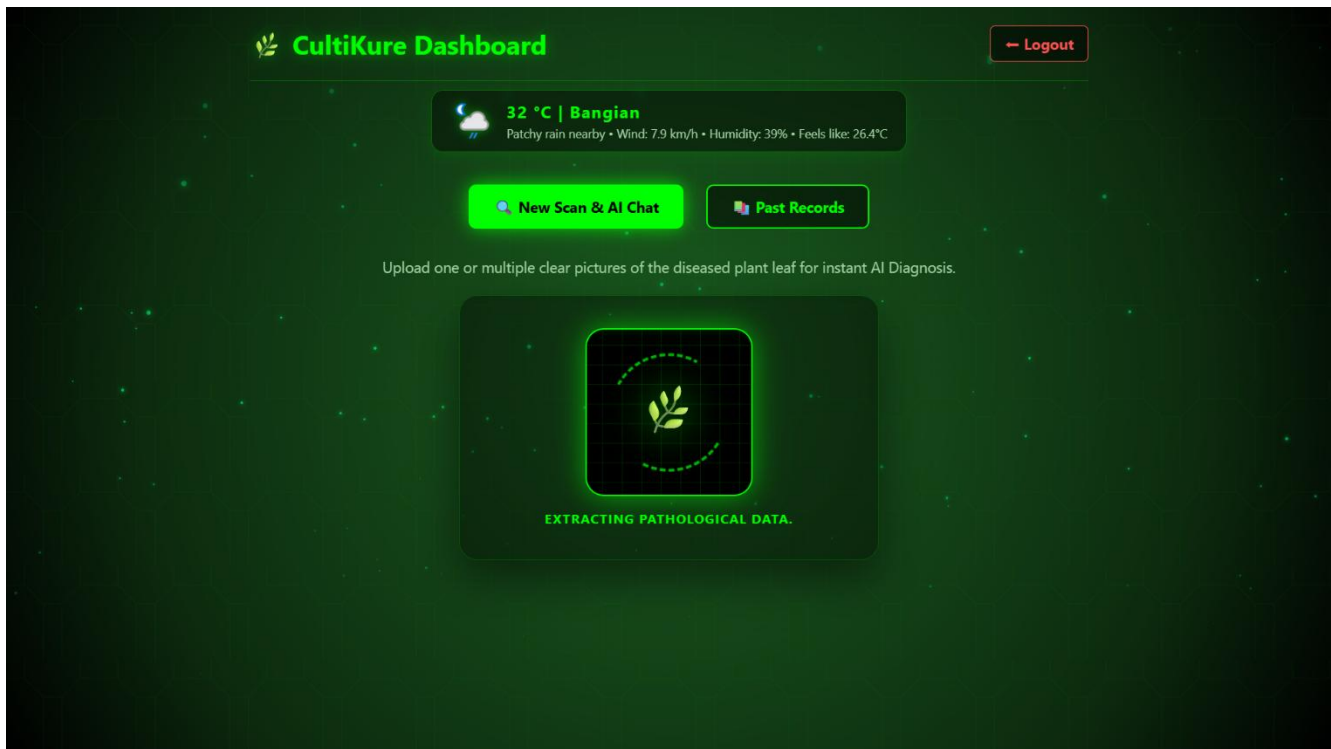
Screen 3 - Signup Page



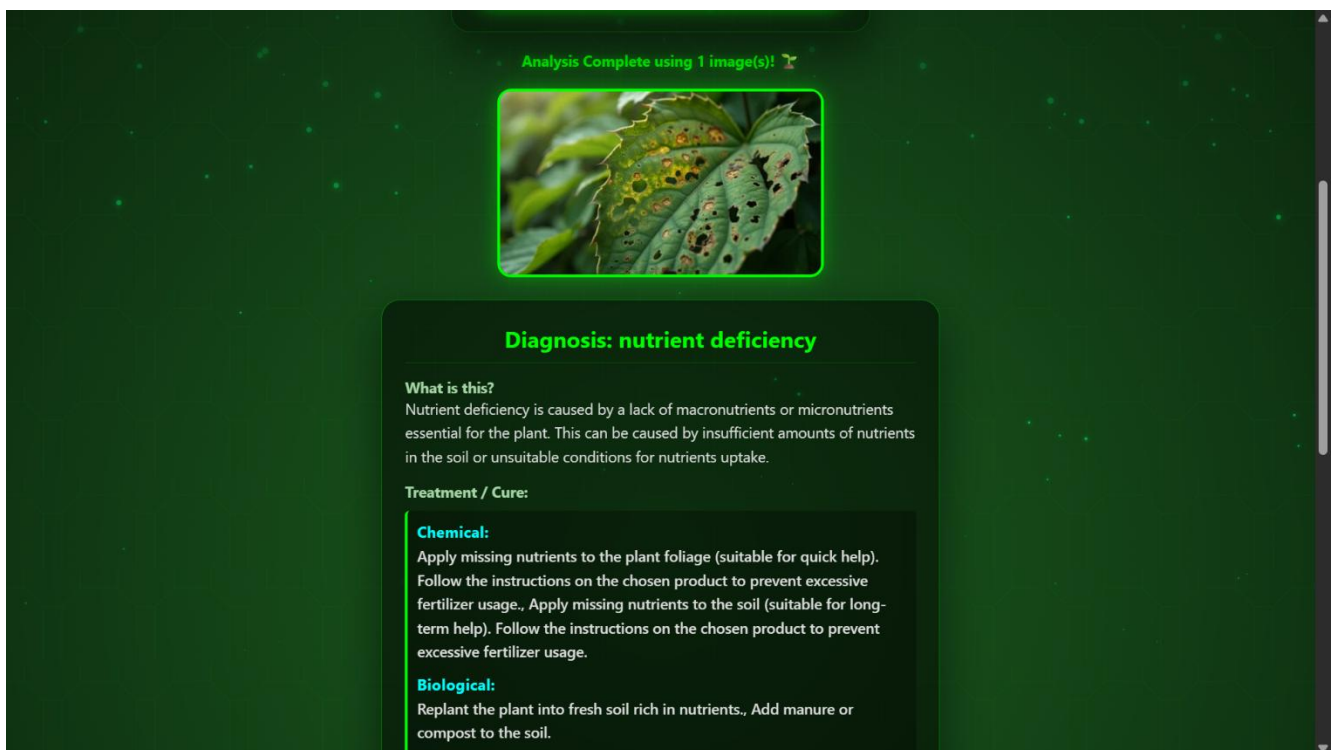
Screen 4 – OTP Validation Page



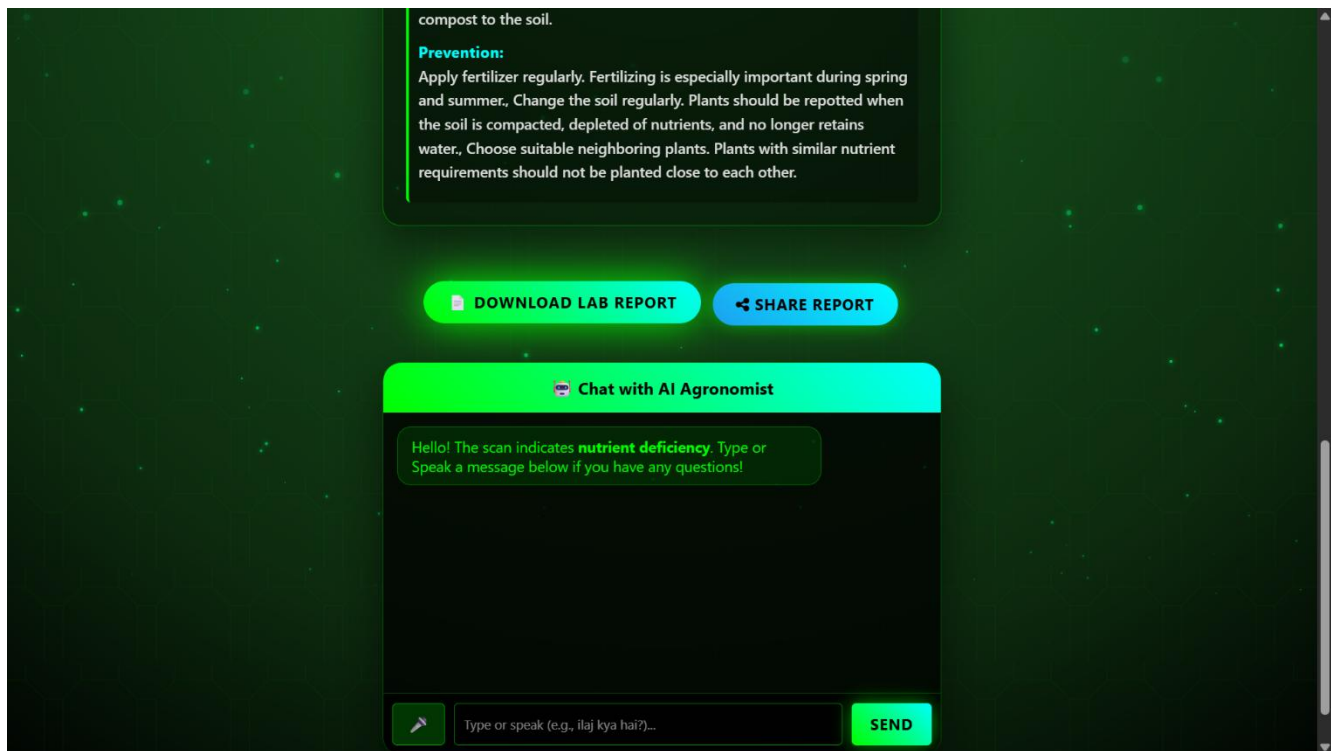
Screen 5 - Dashboard



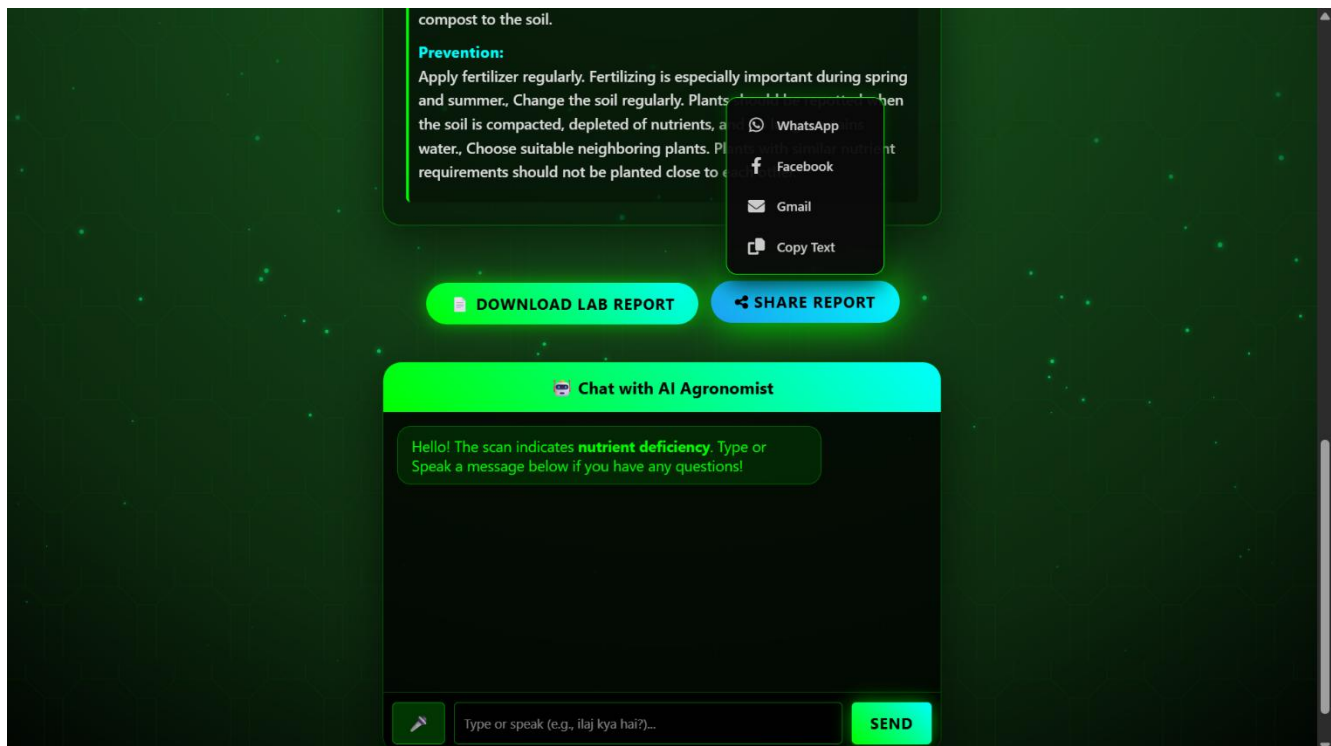
Screen 6 - Analyzation



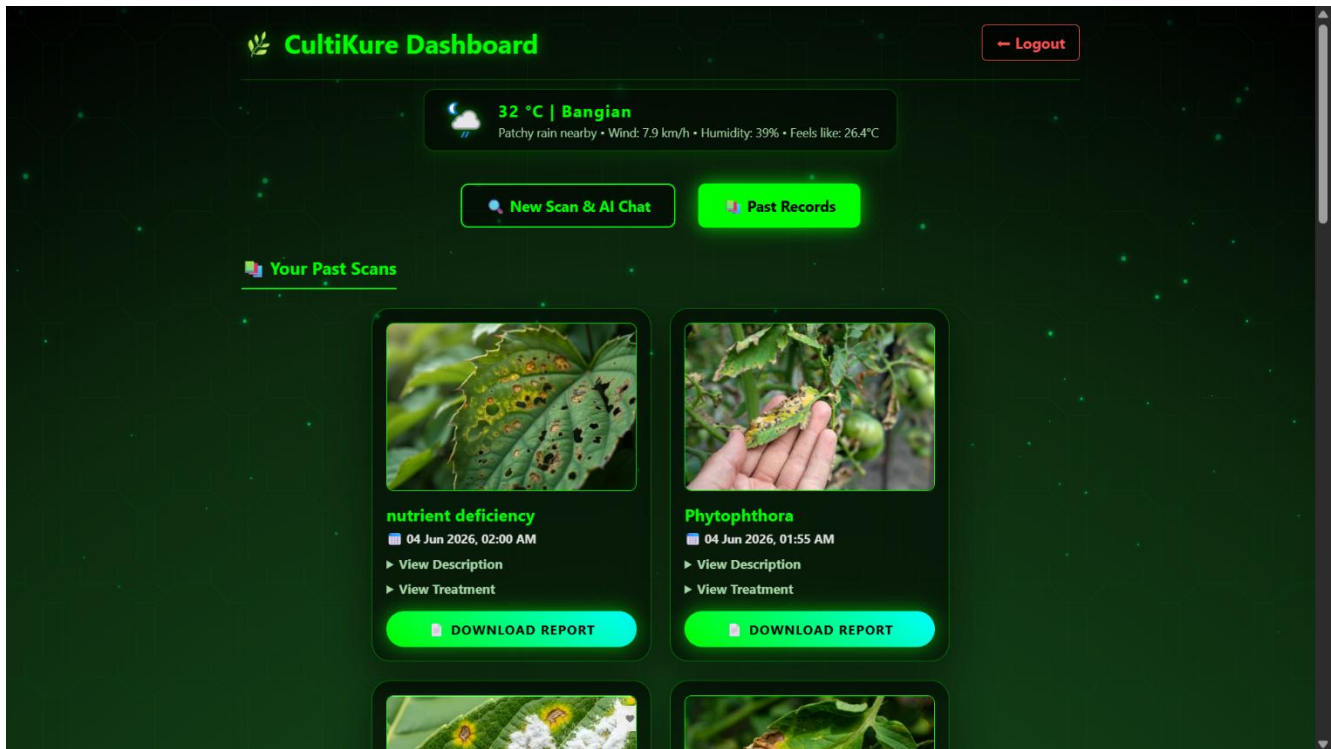
Screen 7 - Results



Screen 8 - Report Generating



Screen 9 – Sharing Report and Chat Bot



Screen – 10 – History (Past Scans)

3.7 Other Design Details

To further optimize performance and reduce server bandwidth, the PDF report generation feature was explicitly designed to execute on the client-side. By integrating the **pdfmake** JavaScript library directly into the frontend UI, the heavy processing required to compile images and text into a high-definition PDF document is handled by the user's local browser memory, ensuring the Flask backend remains unburdened and highly responsive.

Chapter 4. Test Specification and Results

4.1 Test Case Specification

Identifier	TC-1
Related requirements(s)	FR-01 (User Registration & Authentication)
Short description	Verify that the system generates, sends, and verifies an OTP for new user registration.
Pre-condition(s)	User is on the registration page and possesses a valid, accessible email address.
Input data	Valid Email Address, 6-digit OTP code.
Detailed steps	1. Navigate to Sign-Up. 2. Enter email and password. 3. Submit form.

	4. Check email for OTP. 5. Enter OTP on verification screen. 6. Click "Verify".
Expected result(s)	System successfully verifies the OTP and redirects the user to the login/dashboard page.
Post-condition(s)	User's data and hashed password are permanently saved in the SQLite USER table.
Actual result(s)	OTP verified successfully, user redirected to the dashboard.
Test Case Result	Pass

Table 7: TC-1

Identifier	TC-2
Related requirements(s)	FR-02 (Image Upload & Validation)
Short description	Ensure the system securely rejects unsupported file formats during the upload process.
Pre-condition(s)	User is logged into the CultiKure dashboard.
Input data	A .pdf or .txt file (non-image format).
Detailed steps	1. Click on the "Upload Image" portal. 2. Browse and select a .pdf file instead of an image. 3. Attempt to submit the file for analysis.
Expected result(s)	The system rejects the file and immediately displays an "Invalid File Type" warning to the user.
Post-condition(s)	The file is blocked and not sent to the backend or external APIs.
Actual result(s)	Error message displayed on the frontend, upload blocked.
Test Case Result	Pass

Table 8: TC-2

Identifier	TC-3
Related requirements(s)	FR-03 (Computer Vision Disease Detection)
Short description	Validate the API communication between the Flask backend and the Plant.id engine.
Pre-condition(s)	User has successfully uploaded a valid .jpg or .png image of a plant leaf.
Input data	Valid image file (under 5MB).
Detailed steps	1. Upload valid image. 2. Click the "Analyze" button. 3. Backend encodes the image to Base64 and sends a POST request to Plant.id.
Expected result(s)	Backend receives a JSON response containing the exact disease name and a confidence score.
Post-condition(s)	The detected disease name is held in memory to be passed to the next API.

Actual result(s)	Accurate disease name and probability score retrieved in under 3 seconds.
Test Case Result	Pass

Table 9: TC-3

Identifier	TC-4
Related requirements(s)	FR-04 (Generative AI Treatment Plan)
Short description	Verify that Google Gemini API returns a structured, human-readable treatment plan.
Pre-condition(s)	Plant.id API has successfully identified the disease and returned its name.
Input data	Disease Name string (e.g., "Apple Scab").
Detailed steps	1. Flask backend injects the disease name into a strict, pre-defined text prompt. 2. Backend sends the prompt via POST request to Google Gemini API. 3. Await response.
Expected result(s)	Gemini API returns structured text outlining both organic and chemical cures specifically for that disease.
Post-condition(s)	The complete dataset (Image path, Disease, Cure, Date) is saved to the SCAN_HISTORY table.
Actual result(s)	Detailed and perfectly structured treatment plan generated accurately.
Test Case Result	Pass

Table 10: TC-4

Identifier	TC-5
Related requirements(s)	NFR-01 (System Reliability & Error Handling)
Short description	Ensure the system gracefully handles external API failures or network timeouts without crashing.
Pre-condition(s)	User attempts to scan an image.
Input data	Valid leaf image.
Detailed steps	1. Temporarily disable the server's internet connection or input an invalid API key. 2. Upload an image and click "Analyze".
Expected result(s)	The Try-Except blocks catch the error, preventing a server crash, and display a "Service Unavailable" message to the user.
Post-condition(s)	The system remains stable, responsive, and ready for future requests once the connection is restored.
Actual result(s)	Friendly error message displayed. The Flask server did not crash.
Test Case Result	Pass

Table 11: TC-5

Identifier	TC-6
Related requirements(s)	FR-05 (Client-Side Report Generation)
Short description	Validate that the pdfmake JavaScript library generates and downloads a PDF correctly.
Pre-condition(s)	User has completed a scan and is viewing the final result card on the dashboard.
Input data	User mouse click event.
Detailed steps	1. Scroll to the bottom of the result card. 2. Click the "Download Lab Report" button.
Expected result(s)	The browser utilizes local memory to compile the data into a PDF and prompts a file download instantly.
Post-condition(s)	A high-definition PDF file is successfully saved to the user's local device storage.
Actual result(s)	PDF compiled and downloaded successfully without any server-side delay.
Test Case Result	Pass

Table 12: TC-6

4.2 Summary of Test Results

Module Name	Test cases run	Number of defects found	Number of defects corrected so far	Number of defects still need to be corrected
Authentication Module (OTP Generation, Sign-Up, Session Handling)	TC-1	1	1	0
Interface & Validation Module (Image format checks, Glassmorphism UI, PDF generation)	TC-2, TC-6	2	2	0
API Orchestration Module (Plant.id Vision, Gemini Generative AI, Error Handling)	TC-3, TC-4, TC-5	1	1	0
Complete System	<Sum all of the above>	<Sum all of the above>	<Sum all of the above>	<Sum all of the above>

Table 13: Summary of All Test Results

Chapter 5. Conclusion and Future Work

5.1 Project summary

The proposed solution, CultiKure, successfully addresses the persistent problem of delayed and inaccurate plant disease identification in the agricultural sector. By providing an API-first, responsive web application, the system eliminates the hardware and technical barriers associated with native mobile apps, allowing rural farmers to access expert-level diagnostics on low-end devices. To prove the solution's effectiveness, rigorous evaluations were undertaken, including component testing of the image validation modules and integration testing of the external APIs. The results demonstrated that the integration of the Plant.id Convolutional Neural Network yields a high confidence score for disease detection. Furthermore, the Google Gemini API consistently provided accurate, context-aware, and structured generative treatment plans (both organic and chemical). User Acceptance Testing (UAT) confirmed that the system dramatically reduces the diagnostic turnaround time to under 10 seconds, proving that the solution provides real-time, evidence-based agricultural support.

5.2 Problems faced and lessons learned

During the development of this project, several challenges were encountered across different domains:

- **Technical Problems:** Integrating multiple asynchronous APIs (Plant.id, Gemini, and Weather) within a single Flask route occasionally caused server timeouts. Handling the complex JSON data structures returned by the generative AI required strict prompt engineering to prevent data hallucination.
- **Financial Problems:** Acquiring enterprise-grade AI diagnostic capabilities on a student budget was a challenge. This was resolved by utilizing free-tier API limits and optimizing backend requests to prevent redundant API calls.
- **Motivational Problems:** Prolonged debugging sessions, particularly dealing with frontend asynchronous JavaScript and backend state management, were exhausting. However, recognizing the real-world impact of the application kept the momentum alive.

5.3 Future work

To further enhance the CultiKure ecosystem and prepare it for large-scale enterprise deployment, the following future directions are recommended:

- **Database Scalability:** Migrating from the current SQLite architecture to a distributed NoSQL cloud database (such as Firebase Firestore) to handle concurrent read/write operations without locking issues.
- **Multilingual Localization:** Integrating dynamic translation services to provide generative AI prescriptions and UI elements in regional languages, further bridging the communication gap for local farmers.
- **Offline Accessibility:** Developing an SMS/USSD-based integration where farmers without active internet connections can send text-based symptom descriptions and receive automated generative remedies.

References

Journals

Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016), *Using deep learning for image-based plant disease detection*, *Frontiers in Plant Science*, 7 (1419): 1-10. Singh, A. K., Ganapathysubramanian, B., Sarkar, S., & Singh, A. (2018), *Deep learning for plant stress phenotyping: trends and future perspectives*, *Trends in Plant Science*, 23 (10): 883-898.

Books

Goodfellow, I., Bengio, Y., & Courville, A. (2016), *Deep Learning*, MIT Press, Cambridge, 321-350. Grinberg, M. (2018), *Flask Web Development: Developing Web Applications with Python*, O'Reilly Media, Sebastopol, 15-40.

Reference from Internet

Google, *Gemini API Documentation*. <https://ai.google.dev/docs>
Pallets Projects, *Flask Web Development Framework Documentation*.
<https://flask.palletsprojects.com/>
Plant.id, *Plant Disease Identification API Documentation*. <https://github.com/flowerchecker/Plant-id-API>
Zampetti, E., *pdfmake Client-Side PDF Printing*. <https://pdfmake.github.io/docs/>

Appendix A Glossary

- **API (Application Programming Interface):** A set of defined protocols and tools that allows different software applications to communicate with each other (e.g., CultiKure backend requesting data from Google Gemini).
- **Base64:** An encoding scheme used to convert image or binary data into a readable text string format, allowing it to be safely transmitted over HTTP networks to external APIs.
- **CNN (Convolutional Neural Network):** A highly specialized class of deep neural networks commonly applied to analyzing visual imagery. It is the underlying technology powering the Plant.id disease detection engine.
- **Flask:** A lightweight, modular web framework written in the Python programming language, used as the primary backend server for this project.
- **Generative AI:** A type of artificial intelligence capable of generating new content (text, code, images). In this project, Google Gemini acts as the generative engine to synthesize personalized treatment plans.
- **Glassmorphism:** A modern User Interface (UI) design trend emphasizing a semi-transparent, frosted-glass effect, utilized to make the CultiKure frontend visually accessible and minimalistic.
- **JSON (JavaScript Object Notation):** A lightweight and easy-to-read data-interchange format used for transmitting structured data between the Flask server and external web services.
- **OTP (One-Time Password):** A dynamically generated, single-use 6-digit numeric password dispatched via email to verify user identity during registration.
- **REST (Representational State Transfer):** An architectural style for designing networked applications. CultiKure relies on RESTful HTTP requests (GET, POST) for component communications.
- **SQLite:** A C-language library that implements a small, fast, self-contained, and serverless SQL database engine, used for handling user sessions and scan histories.
- **UAT (User Acceptance Testing):** The final phase of software testing where actual end-users test the software in real-world scenarios to validate its functionality and user interface.

Appendix B Deployment/Installation Guide

1. System Prerequisites

- Python 3.8 or higher installed on the host machine.
- An active internet connection (strictly required for API communications).
- Active API Keys for Google Gemini, Plant.id, and an App Password for Gmail SMTP.

2. Local Deployment Instructions Follow these steps to deploy and run the CultiKure application on a local development environment:

- **Step 1: Extract Project Files** Unzip the complete CultiKure project directory to your desired local drive.
- **Step 2: Access the Terminal** Open Command Prompt (Windows) or Terminal (macOS/Linux) and navigate to the extracted project folder using the `cd` command.
- **Step 3: Create a Virtual Environment** Run the following command to create an isolated Python environment to prevent library conflicts: `python -m venv venv`
- **Step 4: Activate the Virtual Environment**
 - For Windows: `venv\Scripts\activate`
 - For macOS/Linux: `source venv/bin/activate`
- **Step 5: Install Dependencies** Install Flask and all required third-party libraries by executing: `pip install -r requirements.txt`
- **Step 6: Configure Environment Variables** Create a file named `.env` in the root directory. Securely add your API credentials as follows: `GEMINI_API_KEY="your_gemini_key_here"`
`PLANT_ID_API_KEY="your_plant_id_key_here"` `MAIL_PASSWORD="your_gmail_app_password"`
- **Step 7: Initialize the Database** Let the system automatically generate the `database.db` SQLite file upon the first execution, or manually trigger the database initialization script if provided (e.g., `python init_db.py`).
- **Step 8: Run the Flask Server** Execute the main application file to start the server: `python app.py`
- **Step 9: Launch the Application** Open any modern web browser (Chrome, Edge, Safari) and navigate to `[http://127.0.0.1:5000](http://127.0.0.1:5000)` to access the CultiKure dashboard.